

生成臨床 UI の層間整合性を評価する TraceBench-EHR

TraceBench-EHR: Evaluating Cross-Layer Integrity in Generated Clinical Interfaces

平田 蓮

Ren Hirata

要旨 生成 AI による臨床ユーザーインターフェースの増分更新では、画面が描画でき、SQL が実行できても、臨床情報要求、SQL、実行結果、表示部品の対応が壊れる可能性がある。本研究は、EHRSQL-2024 と MIMIC-IV Demo を基に、この層間不整合を決定的に生成・判定する TraceBench-EHR を構築する。8 種類の不整合に対し、局所検査、成果物ごとの契約、グラフによる層間契約、同じ内容を持つ平坦な契約を比較し、検出、特定、修復の能力と限界を評価する。現在までに 50 件の実行可能性確認と予備的な UI 更新評価を完了し、正式評価の比較条件と停止条件を定めた。

1. はじめに

電子カルテには、患者属性、入退院、検査、処方、処置など、診療で参照される情報が複数の表と画面に分かれて蓄積される。医療従事者が必要とする情報は診療場面と確認目的によって変わるため、同じ患者データであっても、表示すべき項目、期間、集約方法、提示形式は一意ではない。タスクに応じて画面を生成・更新する臨床ユーザーインターフェース (UI) は、この情報探索を支える一つの方法である。

近年の生成 UI 研究では、自然言語から UI コードを直接作る方法に加え、タスクや意味を表す中間表現を介して UI を生成・編集する方法が提案されている。Jelly はタスク駆動データモデルを生成・更新し、そのモデルから UI を構成する [1]。Bridging Gulfs は設計意図の意味表現を、人の入力と生成 UI を結ぶ層として利用する [2]。SimStep は複数のグラフを用いて、生成結果の前提を人が段階的に確認・修正できるようにする [3]。これらは、生成物の見た目だけでなく、意図と成果物の間に検査可能な表現を置く意義を示している。

一方、臨床 UI では、画面がもっともらしく見えることだけでは十分でない。たとえば「対象患者の直近の腎機能を表示する」という要求に対し、SQL だけ別患者へ変わる、期間条件だけ外れる、SQL 更新後に古い結果が残る、といった更新があり得る。各成果物を単独で見れば、SQL は読み取り専用で実行可能であり、結果は表形式で、Widget も描画可能である。それでも、要求から表示までの対応は壊れている。このような誤

りを本稿では層間不整合と呼ぶ。

本研究は、生成された臨床 UI の増分更新を対象に、臨床情報要求、SQL、実行結果、表示部品の対応を評価する TraceBench-EHR を構築する。EHRSQL-2024[4] の質問と正解 SQL を MIMIC-IV Clinical Database Demo v2.2[5] 上で実行し、基準となる表示までを決定的に生成する。その上で、8 種類の不整合を注入し、局所検査、成果物ごとの契約、グラフによる層間契約、同じ契約内容を持つ平坦なサイドカーを比較する。

本稿の貢献は次の三点である。第一に、臨床情報要求から表示までを対象とする層間不整合の分類を定める。第二に、不整合を再現可能に生成し、実行可能な独立オラクルで採点するベンチマーク設計を示す。第三に、契約内容と保存形式を分けて比較し、検出だけでなく問題箇所の特定と一回の自動修復を評価する。臨床上の安全性、使いやすさ、認知負荷、臨床転帰は本評価の主張範囲に含めない。

2. 関連研究

2.1 生成・更新可能な UI

生成 UI では、利用者の要求から画面または実行可能なコードを作るだけでなく、生成後の変更をどう扱うかが課題となる。直接生成されたコードを局所的に変更すると、変更対象以外の状態、データ参照、表示規則が意図せず変わる可能性がある。Jelly は、利用者の情報タスクをデータモデルとして保持し、自然言語と直接操作をモデルの変更へ変換する [1]。これにより、UI を作り直すたびにタスク構造が失われる問題へ対処する。

意味中間表現を用いる研究では、利用者が生成モ

デルの解釈を確認しやすくすることが重視される。Bridging Gulfs は、製品、機能、デザインシステム、部品などの意味を階層的に構造化し、入力意図と出力 UI の対応を提示する [2]。SimStep は Concept Graph、Scenario Graph、Learning Goal Graph、UI Graph を設け、人が各段階の仮定を修正する [3]。本研究も中間表現を使うが、グラフを介すること自体を利点とみなさず、情報要求、データ取得、実行結果、表示の契約内容を評価対象にする。

2.2 EHR の Text-to-SQL

EHRSQL は、医療従事者から収集した情報要求を基に、電子カルテを対象とする自然言語質問と SQL の組を提供する。EHRSQL-2024 は MIMIC-IV Demo のスキーマに対応し、回答可能な質問と回答不能な質問を含む共有タスクとして整備された [4]。Text-to-SQL の評価では、質問に対して SQL を正しく生成できるか、回答不能な質問で棄権できるかが中心となる。

TraceBench-EHR は SQL 生成モデルの性能を比較しない。正解 SQL を基準として使用し、その実行結果を Widget まで接続した後の増分更新を扱う。つまり、質問と SQL の一致に加えて、SQL と実行結果、実行結果と表示方式、更新後の各成果物の同期を検査する。EHRSQL を基盤にする理由は、質問テンプレート、SQL、公式の train・validation・test 分割があり、更新候補と評価規則を test 結果を見る前に固定できるためである。

2.3 モデル変換と整合性検査

構造化データとビューの更新を対応付ける問題は、双方向変換でも扱われてきた。Lens の研究は、元データとビューの間の往復変換が満たすべき性質を定式化している [6]。また、完全な正解出力を列挙しにくいモデル変換に対して、不変関係を用いるメタモルフィックテストが提案されている [7]。本研究では完成した JSON の完全一致を求めず、変更すべき内容と保持すべき内容を述語として表す。

一般的なスキーマ検査や描画検査は必要であるが、これらは一つの成果物の妥当性を確認する。層間不整合の検出には、複数の成果物間で同じ患者、項目、期間、集約、接続が維持されたかを確認する必要がある。そこで本研究は、局所検査から層間契約までを同じ候補に適用し、検査範囲の違いを明示する。

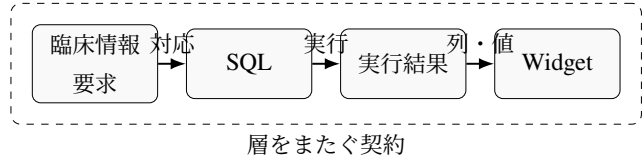


図1 TraceBench-EHR が検査する情報の流れ

3. 研究課題

本研究では、増分更新において最終画面や個別成果物の検査を通過する不整合を対象とする。研究質問を次のように定める。

- RQ1：生成された臨床 UI の増分更新では、局所検査を通過する層間不整合がどの種類で生じるか。
- RQ2：局所検査、成果物ごとの契約、層間契約は、不正更新の検出、妥当更新の受理、問題箇所の特定でどのように異なるか。
- RQ3：検証結果を用いた一回の自動修復は、どの不整合を回復でき、どの不整合で失敗または過剰拒否するか。

想定する仮説は、局所検査では対象患者や期間などの意味的な不整合が通過し、層間契約は不整合の流出を減らしつつ妥当な更新を受理する、というものである。ただし、グラフ契約と同じ情報を平坦なサイドカー契約にも持たせる。両者の差が、契約を持たない条件との差より小さければ、効果はグラフ形式ではなく契約内容に由来すると解釈する。

4. TraceBench-EHR の設計

4.1 基準ケースの構築

入力は EHRSQL-2024 の回答可能な質問と正解 SQL である。回答不能ケースは、正解 SQL から基準 UI を構成できないため主評価から除外し、除外数を報告する。正解 SQL は読み取り専用の SQLite 接続で MIMIC-IV Demo v2.2 へ実行する。SQL ごとの実行上限は 5 秒とし、書き込みを含む文、複数文、許可しない構文は実行前に拒否する。

実行結果が単一値の場合は Metric、表形式の場合は Dataframe を割り当てる。基準ケースは、質問テンプレート、SQL、結果形状、DataNode、Widget の対応を持つ。患者単位の値を契約へ埋め込まず、ケース ID、入力チェックサム、結果形状、対応識別子を保存する。空結果は除外せず、列構造を持つ Dataframe として扱う。

同一の質問テンプレート内にあるケースを更新前後

表 1 TraceBench-EHR の不整合分類

種類	壊す対応
対象患者	要求と SQL が参照する患者
対象項目	要求項目と取得列・集約対象
期間	時点・期間条件と SQL の絞り込み
情報源	必要情報と参照表
集約方法	件数、最大、最小、平均など
表示方式	結果形状と Widget 種別
接続	DataNode と Widget の参照
部分更新	SQL 更新後に残る古い結果

として組み合わせる。これにより、患者、項目、期間などの一部を変えた妥当な更新を作り、同じ基準から不整合候補を生成できる。複数の変異を独立標本として数えず、質問テンプレートを分析上のクラスタとする。

4.2 8 種類の不整合

変異規則は表 1 の 8 種類である。各規則は一つの契約要素を壊し、それ以外の構造を保つ。複数の規則が同じ候補へ偶然混入しないよう、生成後に独立オラクルで検査する。

対象患者の変異では、要求側の患者を保ったまま SQL の患者条件を別の候補へ変える。対象項目では、要求と異なる列または集約対象を取得する。期間では時間表現に対応する WHERE 条件や並び順を壊す。情報源では同じような列名を持つ別表を参照させる。集約方法では、要求された最大値を件数へ変えるなど、SQL として実行可能なまま意味を変える。

表示方式の変異は、SQL、実行結果、列、値を保ち、Metric を Dataframe へ、Dataframe を Table へ変える。局所的には描画可能でも、基準契約が定めた結果形状と表示方式の対応が壊れる。接続の変異は Widget が別の DataNode を参照する。部分更新は SQL だけを新しい要求へ変更し、以前の実行結果を保持する。この変異により、各成果物が個別に妥当でも、更新の同期が失敗する状況を作る。

4.3 独立オラクル

候補生成と正解判定は別のコード経路にする。オラクルは、質問テンプレートから得た患者、項目、期間、情報源、集約と、SQL の解析結果、実行結果の形状、DataNode と Widget の参照を照合する。実行時検査の判定をそのまま正解ラベルとして使わない。候補の期

表 2 候補更新と期待する診断の例

候補	局所状態	層間診断
患者誤り	SQL 実行可能	対象患者
期間欠落	SQL 実行可能	期間
結果残留	表示可能	更新世代
参照誤り	描画可能	接続
表示変更	描画可能	表示方式

待ラベルと独立オラクルが一致しない場合は、正式評価を停止する。

完全な UI JSON の一致を正解としない理由は、意味を保った複数の表現を許容するためである。基準と異なる安定 ID、表示文言、順序であっても、要求、データ、表示の意味条件を満たす候補は妥当とする。一方、見た目が同じでも別患者や古い結果を表示する候補は不整合とする。

4.4 候補と診断の例

基準ケースが「患者 A の直近 24 時間の検査 X を数値で表示する」である場合を考える。妥当な更新として患者 B へ対象を変更するなら、質問側の患者、SQL の患者条件、実行結果の世代、DataNode と Widget の参照を同じ更新として変更する必要がある。一つだけ古い状態に残れば、各成果物を単独では処理できても、更新全体として不整合になる。

対象患者の候補では、SQL 文自体が読み取り専用で、存在する患者を参照していれば局所検査を通過し得る。層間契約は、質問と SQL が指す患者の対応を検査し、対象患者の違反として特定する。期間の候補も同様に、時間条件を削除しても SQL 構文と実行可能性は保たれるため、要求の時間表現との照合が必要になる。

部分更新の候補では、更新後の SQL と更新前の実行結果がそれぞれ妥当な形式を持つ。そこで、SQL のチェックサムと実行結果へ付与した更新世代を対応させ、古い結果が残ったことを診断する。接続の候補では、参照先 DataNode が存在する限り Widget を描画できる場合があるため、Task から期待する DataNode と Widget の辺を検査する。

これらの例では、問題箇所を単に「検証失敗」と返さず、対象患者、期間、更新世代、接続、表示方式のいずれが壊れたかを記録する。この診断単位を修復操作と対応させることで、検出できた候補のうち、期待値が一意に決まる不整合だけを一回の自動修復へ送る。

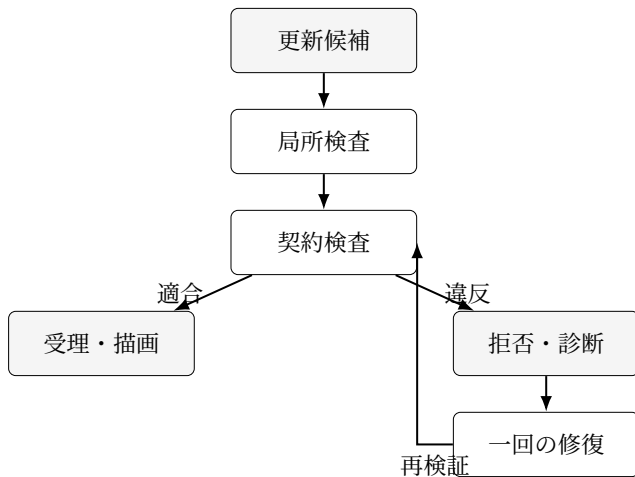


図2 候補検証と一回の契約ベース修復

5. システム構成

提案システムは ScenarioGraph を中心に、Scenario、Task、Data、Widget のノードを持つ。Scenario は診療場面、Task は確認または判断準備、Data は取得する EHR 情報と SQL、Widget は提示形式と配置を表す。基準ケースでは、EHRSQL の一つの質問を一つの Task に対応させ、正解 SQL を持つ DataNode と結果形状に対応する Widget を接続する。

候補更新を受け取ると、検査器はスキーマ、読み取り専用 SQL、SQL 実行、結果形状、描画可能性を確認する。層間契約を持つ条件では、さらに患者、項目、期間、情報源、集約、表示方式、接続、更新世代を照合する。違反がある場合は、候補を画面へ反映せず、層と契約項目を含む診断を返す。

自動修復は、この診断を一回だけ利用する。患者条件、期間条件、Widget 参照など、期待値が契約から一意に決まる場合は該当成果物を書き換える。意味が一意でない場合や、複数層にまたがる変更で安全な修復を決められない場合は拒否する。反復的に試行して正解へ近づける方法は採らず、一回の契約ベース修復として成功、失敗、誤修復を区別する。

6. 実装と再現性

TraceBench-EHR の評価処理は、基準ケースの構築、候補生成、条件別検証、独立オラクルによる採点、統計集計に分ける。基準ケースの構築では、EHRSQL の質問と正解 SQL を読み込み、読み取り専用接続で SQL を実行する。候補生成では、同じ質問テンプレートに属するケースを組み合わせ、固定した変異規則を一つづ

表3 正式評価で保存する成果物

成果物	保存内容
pair manifest	ケース ID、変異種類、チェックサム
candidate results	条件別の受理、特定、修復結果
build summary	基準実行、空結果、変異別件数
summary	主要・副次評価と 95% 区間
report	原稿向けの集計と失敗例
run manifest	入力、設定、コード、出力の版

つ適用する。ここまでの処理は乱数に依存させず、同じ入力と設定から同じ候補チェックサムを得る。

条件別検証は、生成された同一候補を 4 条件へ適用する。各条件は候補を変更せず、利用する検査範囲だけを変える。検証結果は、受理・拒否、違反を特定した層、契約項目、修復操作、検証時間を記録する。独立オラクルは候補生成器と検査器の内部状態を参照せず、保存された候補と基準契約から期待ラベルを再計算する。これにより、候補を作った規則の誤りが、そのまま正解判定へ流れ込むことを避ける。

正式評価は CPU だけで実行し、Gemini を含む外部モデルを呼ばない。seed は 20260827、SQL ごとの上限は 5 秒、ブートストラップは 10,000 回とする。実行時刻は計測値として保存するが、候補の同一性を表すチェックサムには含めない。入力データ、設定、コード、候補、集計のチェックサムを実行マニフェストへ記録し、別の出力先へ再実行した場合に候補と集計が一致するかを確認する。

保存する成果物を表 3 に示す。pair manifest には候補を再識別する最小情報を置き、candidate results には条件別の判定を一実行一行で記録する。summary は質問テンプレート単位の集計から主要・副次評価を作り、report は原稿で確認しやすい形へ要約する。run manifest はデータセット commit、コード commit、設定版、実行環境、出力チェックサムを結び付ける。

テストでは、8 種類の変異が意図した一つの契約だけを壊すこと、妥当候補が独立オラクルを満たすこと、グラフ契約と同じ内容のサイドカー契約が同じ判定を返すこと、修復後に再検証が行われることを確認する。さらに、質問文、SQL、患者 ID、結果値を成果物へ書き出そうとした場合に停止する検査を設ける。機能テスト、lint、型チェックがすべて成功しない限り評価版

表 4 比較する検査条件

条件	検査範囲
局所検査	スキーマ、SQL 安全性、実行、描画
成果物契約	各成果物が持つ患者・項目など
グラフ契約	質問から Widget までの層間対応
サイドカー契約	グラフと同じ対応を平坦に保持

を固定しない。

再現性の対象は、許可された入力から候補、判定、集計を再生成できることまでである。MIMIC 由来の生データや患者単位の結果をリポジトリへ含めないため、第三者が再実行する場合は、各データセットの利用条件に従って入力を取得する必要がある。公開可能な成果物と再配布できない入力を分け、どの版を使ったかをチェックサムで検証できる構成にする。

7. 評価方法

7.1 比較条件

同じ基準ケースと候補を表 4 の 4 条件へ適用する。条件間で候補生成、SQL 実行環境、一般的なスキーマ検査をそろえる。

局所検査は、各成果物が単独で実行可能かを確認する。成果物ごとの契約は、SQL、実行結果、Widget に期待する属性を個別に持つが、層をまたぐ識別関係を持たない。グラフ契約は、質問、DataNode、SQL、実行結果、Widget を辺で結ぶ。サイドカー契約は同じ契約内容を、グラフ構造を使わない平坦な対応表として保持する。

グラフ契約とサイドカー契約を比較に含めることで、契約内容の効果と保存形式の効果に分ける。両条件の性能が一致した場合、グラフの一般的な優位性は主張しない。グラフは影響範囲の可視化や編集単位として別の利点を持ち得るが、それは本技術評価だけでは確定しない。

7.2 評価指標

主要評価は**不整合の流出率**と**妥当な更新の受理率**である。不整合の流出率は、不整合候補のうち受理されて UI へ反映可能になった割合であり、低いほどよい。妥当な更新の受理率は、正しい候補のうち SQL 実行と描画まで完了して受理された割合であり、高いほどよい。二つを一つの正解率にまとめないのは、安全な拒否を

増やすだけで不整合の流出を下げる条件を区別するためである。

副次評価は、問題箇所の特定期率、一回の修復成功率、誤修復率、過剰拒否率、検証時間である。問題箇所は不整合の層と契約項目の組で採点する。修復成功は、修復後の候補が独立オラクルを満たし、SQL 実行と描画まで完了することと定義する。拒否すべきケースを無理に修復した場合は誤修復として分ける。

条件差と 95% 信頼区間は、質問テンプレート単位のクラスターブートストラップ 10,000 回で求める。同じ質問から作った複数の変異を独立標本として扱わない。平均だけでなく、変異種類別の分布と代表的な失敗例を報告する。

7.3 分割と停止条件

EHRSQL-2024 の train で実装とテストを作り、validation の回答可能 931 件でパイロットを行う。パイロット通過後に変異規則、独立オラクル、比較条件、分析コードを commit で固定する。その後、test の回答可能 934 件を一度だけ実行する。test 開始後に評価規則の変更が必要になった場合は実行を停止し、その結果を正式評価として使用しない。

ほかの停止条件は、入力チェックサムの不一致、テスト・lint・型チェックの失敗、基準 SQL 実行またはグラフ検証成功率が 95% 未満、各変異が 10 テンプレートまたは 30 候補未満、保存成果物への質問文・SQL・患者 ID・結果値の混入である。停止条件を先に定めることで、評価結果を見た後の規則変更を抑える。

8. 現在までの結果

8.1 50 件の実行可能性確認

2026 年 8 月 27 日に、train 分割の回答可能ケースから異なる質問テンプレートを優先して 50 件を選び、正解 SQL の実行と最小 ScenarioGraph への変換を確認した。結果を表 5 に示す。

全 50 件で正解 SQL を実行でき、ScenarioGraph の検証に成功した。49 件で非空結果を取得し、1 件は列を持つ空結果だった。結果形状に基づき 40 件を Metric、10 件を Dataframe へ割り当てた。タイムアウト、SQL エラー、書き込み拒否、グラフ検証エラーはなかった。この結果は、選定した 50 テンプレートを現行実装へ機械的に変換できることを示す。全ケースへの適用可能性や表示方式の臨床的妥当性を示すものではない。

表 5 EHRSQL 50 件の実行可能性確認

確認項目	結果
正解 SQL の実行成功	50/50
非空結果	49/50
ScenarioGraph 検証成功	50/50
Metric への割当	40/50
Dataframe への割当	10/50
タイムアウト・書込拒否・SQL エラー	0

初回の成果物では、SQLite が返す列名に SQL 断片が含まれるケースがあった。この出力を無効とし、列名を成果物へ保存しないよう修正して再実行した。最終成果物には質問文、SQL、患者 ID、患者単位の結果値が含まれないことを確認した。この事例から、評価指標だけでなく、生成物への機微情報の混入を停止条件として機械的に検査する必要が分かった。

8.2 UI 更新技術評価 v0.4

TraceBench-EHR に先立つ予備評価として、合成した 24 ケースと 120 件の共通変更仕様を用い、同じ更新要求と安全要件を持つ直接差分方式と完全グラフ方式を比較した。各ケースは妥当候補 1 件、単一違反 3 件、複合違反 1 件を持つ。比較中に生成モデルを呼ばず、候補は決定的に作成した。

直接差分方式と完全グラフ方式は、ともに違反候補 96 件をすべて拒否し、妥当候補 24 件をすべて受理した。違反流出率と妥当更新受理率の対応差は 0.0 で、95% 区間も 0.0 から 0.0 だった。グラフ方式から変更対象外の保護、安全条件、追跡検査を個別に外した条件では、対応する違反 18 件が通過した。

この結果は、同じ契約内容を実装した範囲で、グラフ形式そのものが直接差分より高い検出性能を持つという仮説を支持しなかった。一方、各制御を外したときに対応する違反が通過したため、更新範囲、安全条件、追跡関係を明示的に検査する役割は確認できた。そこで正式評価では、グラフ方式に弱い基準だけを置かず、同じ契約内容を持つ平坦なサイドカーを強い比較条件にする。

8.3 validation 初回パイロット

正式評価 v1.0 の初回パイロットでは、validation 全 1,163 件のうち正解 SQL を持つ 931 件を対象とした。

SQL 実行は 924 件で成功し、成功率は 99.25% だった。625 組、5,000 候補を評価したが、当初予定した列対応の変異は候補 0 件、テンプレート 0 件となった。validation で実行に成功した SQL 結果がすべて 1 列であり、列順を変えて意味的な対応を壊す候補を構成できなかったためである。

この結果は事前の最小件数を満たさないため、v1.0 を固定せず、列対応を表示方式の対応へ置き換えた v1.1 へ更新した。表示方式の変異は、SQL、実行結果、列、値を保ったまま Metric、Dataframe、Table の対応だけを変える。他の 7 種類の変異、4 比較条件、主要・副次評価、分析方法、seed、データ管理、停止条件は維持する。v1.1 で validation を再実行し、停止条件を通過した場合に限って固定し、test を一度だけ実行する。

9. 考察

9.1 局所妥当性と層間整合性

50 件の実行可能性確認では、正解 SQL を ScenarioGraph と Widget へ変換できた。一方、これは各層を正しく接続した基準ケースの確認であり、増分更新後に対応が保たれることを保証しない。SQL が実行できること、結果が表であること、Widget が描画できることは、すべて必要条件である。しかし、患者、項目、期間、集約、表示の意味が一致しているかは別の検査を要する。

層間不整合は、最終画面の見た目から判別しにくい場合がある。別患者の値でも数値カードとして自然に表示され、古い結果でも異常なく描画される可能性がある。したがって、画面のスクリーンショット比較や UI コードの構文検査だけで、情報要求に対する正しさを評価することは難しい。本研究の契約は、画面に至るまでの根拠を機械可読に保持し、どの対応が壊れたかを診断できるようにする。

9.2 契約内容とグラフ形式

予備評価 v0.4 で直接差分方式と完全グラフ方式に差がなかったことは、グラフを用いれば自動的に整合性が得られるわけではないことを示す。同じ要件を成果物側に実装できるなら、検出性能が同じになる可能性がある。そこで TraceBench-EHR は、グラフと同じ情報を持つサイドカー契約を比較する。

グラフ形式が有用である可能性は、契約の再利用、影響範囲の表示、編集履歴、複数タスクで共有するデー

タの表現などに残る。ただし、これらは検出率だけでは測れない。検査の実装箇所、変更時に更新する箇所、契約の重複、人が問題箇所を理解するための時間などを別に評価する必要がある。本稿では、保存形式としての JSON、関係表、グラフデータベースの一般的な優劣を主張しない。

9.3 自動修復の範囲

契約から期待値が一意に決まる不整合は、自動修復に適している。たとえば Widget が参照すべき DataNode、SQL 更新後に対応する結果世代、基準ケースで定めた Widget 種別は、診断から修復操作を構成できる。一方、自然言語要求が複数の解釈を許す場合や、表示粒度と臨床的な意味の選択が必要な場合、技術的な契約だけで正解を決められない。

そのため、修復率を高くすることだけを目的にしない。誤修復と過剰拒否を分け、安全に一意化できない場合は拒否する。将来、人による評価を行う場合は、成果物の差分だけを提示する条件と、グラフから得た影響範囲、根拠、保護条件を提示する条件を比較し、専門家が誤りを検出・修正できるかを検討する。

9.4 評価設計から得た知見

validation 初回パイロットで列対応の変異を作れなかったことは、故障モデルを概念だけで固定せず、実データの結果形状で実行可能性を確認する必要性を示した。変異規則を test で初めて試すと、正式評価の途中で規則変更が必要になる。train で実装し、validation で停止条件を確認し、test を一度だけ実行する手順は、この事後変更を抑えるために重要である。

また、質問テンプレート単位で集計する必要がある。同じテンプレートから患者や項目だけを変えたケースは、SQL 構造と不整合生成規則を共有する。候補単位で独立に数えると、特定のテンプレートが結果へ過大に寄与する。クラスターブートストラップと変異種類別の報告により、評価単位の依存を明示する。

10. 制約と倫理

本研究は MIMIC-IV Clinical Database Demo v2.2 を用いる技術評価である。評価から、臨床上の安全性、診療判断の正確さ、使いやすさ、認知負荷、臨床転帰、実運用、他施設への一般化を主張しない。正解 SQL は技術的な基準であり、表示内容の医学的な妥当性を専門家が確認した基準ではない。

表 6 今後の作業

時期	作業
直近	v1.1 の validation 再実行と停止条件確認
固定後 評価後	test 934 件の一回実行と別経路集計結果、信頼区間、失敗例、修復例の整理
原稿化 修士論文	図表、再現条件、倫理・制約の確定 技術評価と将来の人対象評価を区別して統合

人を対象とする評価は初回の主評価から外している。既存の専門家評価案は、倫理審査と実施許可の範囲、募集方法、データ保存、同意、撤回方法が確認されるまで実施しない。KLM による予備的な操作数の確認も、読解、見落とし、判断、認知負荷を示す結果として扱わない。

生データ、質問文、正解 SQL、患者 ID、患者単位の結果値は GitHub と Notion へ保存しない。Git 管理外の一時領域だけで扱い、公開成果物にはケース ID、入力と候補のチェックサム、変異種類、判定結果、集計、実行マニフェストを保存する。MIMIC 由来の派生成果物に関する利用条件を確認し、データセットのライセンスと出典を明記する。

11. 今後の予定

修士論文までの作業を表 6 に示す。直近では v1.1 の validation 再実行を行い、停止条件、成果物への機微情報混入、再現性を確認する。通過後に設定、コード、分析を commit で固定し、test の回答可能 934 件を一度だけ実行する。

正式評価後は、変異種類ごとの流出率、受理率、特定率、修復率を整理する。グラフ契約とサイドカー契約が一致した場合は契約内容の効果として解釈し、不一致がある場合は実装差や表現上の差を調査する。評価規則を変更する必要が生じた場合は版を上げ、test 結果を見た後の変更であることを明示する。

修士論文では、プロトタイプシステムの設計、UI 更新予備評価、TraceBench-EHR の設計と正式評価を一つの流れとして整理する。人対象評価は、技術評価で検出・特定・修復の範囲を確立した後の課題として分ける。臨床 UI の利用効果を扱う場合は、専門家確認済

みの症例と採点基準を用い、タスク成功率、成功試行の完了時間、操作数、NASA-TLX、SUS を別々に報告する。

12. まとめ

本研究は、生成臨床 UI の増分更新で、臨床情報要求、SQL、実行結果、表示部品の対応が壊れる層間不整合を扱う。TraceBench-EHR は EHRSQL-2024 と MIMIC-IV Demo を基に基準 UI を構成し、8 種類の不整合を定期的に注入する。局所検査、成果物ごとの契約、グラフ契約、同じ内容を持つサイドカー契約を同じ候補で比較し、検出、妥当更新の受理、問題箇所の特定、一回の修復を評価する。

現在までに、異なる 50 テンプレートで正解 SQL の実行と ScenarioGraph への変換を確認した。予備評価では、同じ契約内容を持つ直接差分方式とグラフ方式の検出性能に差がなく、グラフ形式だけでは優位性を説明できなかった。正式評価の初回パイロットでは列対応の変異を作れないことが分かり、test 実行前に表示方式の変異へ改訂した。今後は validation で v1.1 を固定し、test を一度だけ実行して、層間契約と自動修復の能力と限界を明らかにする。

参考文献

- [1] Yining Cao, Peiling Jiang, and Haijun Xia. Generative and malleable user interfaces with generative and evolving task-driven data model. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*, pp. 1–20, 2025.
- [2] Seokhyeon Park, Soohyun Lee, Eugene Choi, Hyunwoo Kim, Minkyu Kweon, Yumin Song, and Jinwook Seo. Bridging gulfs in UI generation through semantic guidance. In *Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems*, pp. 1–16, 2026.
- [3] Zoe Kaputa, Anika Rajaram, Vryan Feliciano, Zhuoyue Lyu, Maneesh Agrawala, and Hariharan Subramonyam. SimStep: Human-in-the-loop authoring of interactive educational simulations through task-level abstractions. In *Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems*, pp. 1–34, 2026.
- [4] Gyubok Lee, Sunjun Kweon, Seongsu Bae, and Edward Choi. Overview of the EHRSQL 2024 shared task on reliable text-to-SQL modeling on electronic health records. In *Proceedings of the 6th Clinical Natural Language Processing Workshop*, pp. 644–654. Association for Computational Linguistics, 2024.
- [5] Alistair Johnson, Lucas Bulgarelli, Tom Pollard, Steven Horng, Leo Anthony Celi, and Roger Mark. MIMIC-IV. *PhysioNet*, 2023. Version 2.2.
- [6] J. Nathan Foster, Michael B. Greenwald, Jonathan T. Moore, Benjamin C. Pierce, and Alan Schmitt. Combinators for bidirectional tree transformations: A linguistic approach to the view-update problem. *ACM Transactions on Programming Languages and Systems*, Vol. 29, No. 3, 2007.
- [7] Xiao He, Zhenjiang Li, Zhenjiang Hu, Xiaoyu Liu, and Takashi Hirayama. Testing bidirectional model transformation using metamorphic testing. *Information and Software Technology*, Vol. 104, pp. 48–69, 2018.